

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyze	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	BL5 – Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze

8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

*I **Kadhirezhilan. D [192511228]** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1. Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used.

Aim

To retrieve specific information from multiple related tables using SQL JOIN and Subquery, and explain how each query works in a relational database.

Relations [Student]

StudentID	Name	DeptID
101	Rahul	D01
102	Priya	D02
103	Arjun	D01

Department

DeptID	DeptName
D01	CSE
D02	ECE

SQL Query using JOIN

Problem

Display the **Student ID, Student Name, Department Name and Marks** by combining the Student and Department tables.

```
mysql> SELECT S.StudentID,
->      S.Name,
->      D.DeptName
-> FROM Student S
-> JOIN Department D
-> ON S.DeptID = D.DeptID;
+-----+-----+-----+
| StudentID | Name   | DeptName |
+-----+-----+-----+
| 101       | Rahul  | CSE      |
| 103       | Arjun  | CSE      |
| 105       | Karthik| CSE      |
| 102       | Priya  | ECE      |
| 104       | Sneha  | EEE      |
+-----+-----+-----+
5 rows in set (0.02 sec)
```

Explanation of JOIN

What is a JOIN?

A JOIN is used to combine data from two or more tables based on a common column.

In this example,

- Student table contains student details.
- Department table contains department details.
- The common attribute is **DeptID**.

The database compares the DeptID in both tables and combines matching rows.

Execution Steps

Step 1

Read all records from the Student table.

Step 2

Read all records from the Department table.

Step 3

Compare the DeptID values.

Step 4

Combine matching rows.

Step 5

Display StudentID, Name, Department Name and Marks.

Advantages of JOIN

- Retrieves data from multiple tables simultaneously.
- Eliminates data redundancy.
- Faster than many nested subqueries.
- Improves query performance.
- Widely used in real-world database applications.

Problem

Display the names of all students who belong to the **CSE department**.

```
mysql> SELECT Name FROM Student WHERE DeptID = (SELECT DeptID FROM Department WHERE DeptName = 'CSE');
+-----+
| Name |
+-----+
| Rahul |
| Arjun |
| Karthik |
+-----+
3 rows in set (0.00 sec)
```

Explanation of Subquery

A subquery is a query written inside another SQL query. It is also called an **Inner Query** or **Nested Query**.

Real-World Applications

JOIN

- Student Management System
- Banking System
- Hospital Management
- Employee Payroll
- E-Commerce Applications

Subquery

- Finding highest salary.
- Finding students with highest marks.
- Finding customers with maximum purchases.
- Filtering records using aggregate values.

Result

- Successfully created the Student and Department tables.
- Successfully inserted sample data.
- Successfully retrieved information using **INNER JOIN**.
- Successfully retrieved information using **Subquery**.
- Verified the output and understood the execution logic of both approaches.

Conclusion

SQL **JOIN** and **Subquery** are fundamental techniques for retrieving related information from relational databases. The **INNER JOIN** combines rows from multiple tables using a common attribute, making it highly efficient for generating reports and displaying related data. A **Subquery** executes an inner query first and then uses its result in the outer query, making it useful for filtering and decision-making operations. In this example, the JOIN retrieved complete student and department information, while the Subquery identified students belonging to the CSE department. Therefore, both techniques are essential in database applications, with JOIN generally preferred for performance and Subquery preferred for logical filtering and complex conditions.

Q2. Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations.

Aim

To formulate **Relational Algebra expressions** for retrieving records that satisfy multiple conditions across related relations and convert them into equivalent SQL queries.

Query 1

Retrieve the names of students who belong to the CSE department.

Relational Algebra Expression

$$\pi_{Name} \left(\sigma_{DeptName='CSE'} (Student \bowtie_{Student.DeptID=Department.DeptID} Department) \right)$$

Query 2

Retrieve the names of students who belong to the CSE department and scored more than 80 marks.

Relational Algebra Expression

$$\pi_{Name} \left(\sigma_{DeptName='CSE' \wedge Marks > 80} (Student \bowtie_{Student.DeptID=Department.DeptID} Department) \right)$$

Query 3

Retrieve the Student ID, Name, and Department Name of students who scored more than 85 marks.

Relational Algebra Expression

$$\pi_{StudentID, Name, DeptName} \left(\sigma_{Marks > 85} (Student \bowtie_{Student.DeptID=Department.DeptID} Department) \right)$$

Query 4

Retrieve all students who belong to either the CSE or ECE department.

Relational Algebra Expression

$$\pi_{StudentID, Name, DeptName} \left(\left(\begin{array}{c} \sigma_{DeptName='CSE' \vee DeptName='ECE'} \\ Student \bowtie_{Student.DeptID=Department.DeptID} \\ Department \end{array} \right) \right)$$

Explanation

- **Join ($\bowtie$)** combines the Student and Department relations using the common attribute **DeptID**.
- **Selection (σ)** filters tuples based on one or more conditions.
- **Projection (π)** displays only the required attributes.

Conclusion

Relational algebra provides a mathematical foundation for database queries. By combining **Join**, **Selection**, and **Projection**, complex queries involving multiple conditions across related relations can be expressed clearly and efficiently.

Q3. Given a Real-World Scenario (Student Management System), Design a Relational Schema and Construct Appropriate SQL Queries for Data Retrieval and Updates

Aim

To design a relational database schema for a Student Management System and write SQL queries to retrieve and update data efficiently.

Scenario

A college wants to maintain information about students, departments, and courses. The system should:

- Store student details.
- Store department information.
- Store course details.
- Retrieve student records with department information.
- Update student information whenever required.

Relational Schema

DEPARTMENT

Attribute	Data Type	Constraint
DeptID	INT	Primary Key
DeptName	VARCHAR(50)	NOT NULL

STUDENT

Attribute	Data Type	Constraint
StudentID	INT	Primary Key
Name	VARCHAR (50)	NOT NULL
DeptID	INT	Foreign Key
Marks	INT	CHECK (Marks >= 0)

Foreign Key

DeptID references Department(DeptID).

COURSE

Attribute	Data Type	Constraint
CourseID	INT	Primary Key
CourseName	VARCHAR (50)	NOT NULL
Credits	INT	NOT NULL

SQL Queries

1. Create Department Table

```
mysql> CREATE TABLE Department(  
    -> DeptID INT PRIMARY KEY,  
    -> DeptName VARCHAR(50)  
    -> );  
Query OK, 0 rows affected (0.05 sec)
```

2. Create Student Table

```
mysql> CREATE TABLE Student(  
    -> StudentID INT PRIMARY KEY,  
    -> Name VARCHAR(50),  
    -> DeptID INT,  
    -> Marks INT,  
    -> FOREIGN KEY(DeptID)  
    -> REFERENCES Department(DeptID)  
    -> );  
Query OK, 0 rows affected (0.11 sec)
```

3. Create Course Table

```
mysql> CREATE TABLE Course(  
    -> CourseID INT PRIMARY KEY,  
    -> CourseName VARCHAR(50),  
    -> Credits INT  
    -> );  
Query OK, 0 rows affected (0.05 sec)
```

Insert Records

```
mysql> INSERT INTO Department VALUES  
    -> (1,'CSE'),  
    -> (2,'ECE');  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> INSERT INTO Student VALUES  
    -> (101,'Rahul',1,85),  
    -> (102,'Priya',2,92);  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0  
  
mysql>  
mysql> INSERT INTO Course VALUES  
    -> (201,'Database Management System',4),  
    -> (202,'Computer Networks',3);  
Query OK, 2 rows affected (0.01 sec)  
Records: 2 Duplicates: 0 Warnings: 0
```

Data Retrieval Query

Retrieve student details along with their department names.

```
mysql> SELECT S.StudentID,  
-> S.Name,  
-> D.DeptName,  
-> S.Marks  
-> FROM Student S  
-> JOIN Department D  
-> ON S.DeptID=D.DeptID;
```

StudentID	Name	DeptName	Marks
101	Rahul	CSE	85
102	Priya	ECE	92

2 rows in set (0.02 sec)

Data Update Query

Update Rahul's marks.

```
mysql> UPDATE Student  
-> SET Marks=90  
-> WHERE StudentID=101;  
Query OK, 1 row affected (0.02 sec)  
Rows matched: 1 Changed: 1 Warnings: 0
```

Verify the Update

```
mysql> SELECT * FROM Student WHERE StudentID=101;
```

StudentID	Name	DeptID	Marks
101	Rahul	1	90

1 row in set (0.02 sec)

Explanation

- The Department table stores department details.
- The Student table stores student records and references the Department table using a foreign key.
- The Course table stores available courses.
- The JOIN query retrieves student information along with department names.
- The UPDATE query modifies an existing record while preserving database integrity.

Conclusion

A well-designed relational schema minimizes data redundancy, maintains referential integrity through primary and foreign keys, and supports efficient data retrieval and updates. Using SQL queries such as SELECT and UPDATE, the Student Management System can effectively manage academic information while ensuring data consistency and accuracy.

Q4. Analyse a Given SQL Query with Nested Subqueries and Predict Its Output

Aim

To analyze the execution of a nested SQL query and predict its output by explaining each step of execution.

Sample Table: Student

StudentID	Name	DeptID	Marks
101	Rahul	1	85
102	Priya	2	92
103	Arjun	1	78
104	Sneha	3	88
105	Karthik	1	95

Step-by-Step Execution

Step 1: Execute the Inner Query

```
mysql> SELECT Name, Marks FROM Student WHERE  
      -> Marks = ( SELECT MAX(Marks) FROM Student );  
+-----+-----+  
| Name  | Marks |  
+-----+-----+  
| Priya |    92 |  
+-----+-----+  
1 row in set (0.00 sec)
```

Step 2: Execute the Outer Query

The outer query becomes:

```
mysql> SELECT Name, Marks FROM Student  
      -> WHERE Marks = 95;  
Empty set (0.00 sec)
```

Step 3: Retrieve Matching Record

The database searches the Student table for records where **Marks = 95**.

Explanation

- The query contains a **nested subquery**, where one query is written inside another.
- The **inner query** executes first and returns the highest marks.
- The **outer query** uses this result to retrieve the student(s) whose marks equal the maximum value.
- SQL processes nested subqueries from the innermost query to the outermost query.

Conclusion

Nested subqueries improve SQL query flexibility by allowing one query to depend on the result of another. In this example, the inner query identified the highest marks, and the outer query retrieved the corresponding student record. This method is useful for solving complex database queries involving aggregate functions and conditional filtering.

Q5. Compare Different SQL Approaches (JOIN vs Subquery) for Solving the Same Problem and Evaluate Their Efficiency

Aim

To compare the SQL approaches **JOIN** and **Subquery** for retrieving the same information and evaluate their efficiency in terms of performance, readability, and optimization.

Problem Statement

Retrieve the **Student Name** and **Department Name** for all students.

Relations

Student

StudentID	Name	DeptID
101	Rahul	1
102	Priya	2
103	Arjun	1
104	Sneha	3
105	Karthik	1

Department

DeptID	DeptName
1	CSE
2	ECE
3	EEE

Approach 1 – Using JOIN

SQL Query

```
mysql> SELECT S.StudentID, S.Name, D.DeptName FROM Student S
->
-> INNER JOIN Department D ON S.DeptID = D.DeptID;
+-----+-----+-----+
| StudentID | Name | DeptName |
+-----+-----+-----+
|      101 | Rahul | CSE      |
|      102 | Priya | ECE      |
+-----+-----+-----+
2 rows in set (0.02 sec)
```

Explanation

- INNER JOIN combines the Student and Department tables using the common column DeptID.
- It retrieves all matching records in a single query execution.

Approach 2 – Using Subquery

SQL Query

```
mysql> SELECT StudentID, Name, (SELECT DeptName FROM Department D WHERE D.DeptID = S.DeptID) AS DeptName
-> FROM Student S;
+-----+-----+-----+
| StudentID | Name | DeptName |
+-----+-----+-----+
|      101 | Rahul | CSE      |
|      102 | Priya | ECE      |
+-----+-----+-----+
2 rows in set (0.02 sec)
```

Explanation

- The outer query retrieves student details.
- The subquery executes to fetch the corresponding department name for each student.

Comparison of JOIN and Subquery

Feature	JOIN	Subquery
Execution Method	Combines tables directly	Uses an inner query inside the main query
Performance	Faster for large datasets	Generally slower for large datasets
Readability	Easy for multi-table retrieval	Simple for small filtering tasks
Optimization	Better optimized by the DBMS	May require repeated execution
Best Use	Retrieving related data from multiple tables	Filtering, aggregate functions, and conditional queries

Efficiency Evaluation

- JOIN is more efficient because the database optimizer processes related tables in a single execution plan.
- Subqueries are easier to understand for simple problems but can become slower when executed repeatedly for many rows.
- For large databases, JOIN is generally preferred because it provides better performance and scalability.

Advantages

JOIN

- Faster execution.
- Better performance on large databases.
- Retrieves data from multiple tables efficiently.
- Optimized by most relational database management systems.

Subquery

- Easy to write for simple conditions.
- Useful with aggregate functions such as MAX(), MIN(), and AVG().
- Improves logical separation of complex conditions.

Conclusion

Both JOIN and Subquery can solve the same database problem and produce identical results. However, JOIN is generally more efficient because it executes faster, is better optimized by the DBMS, and is more suitable for retrieving data from multiple related tables. Subqueries are useful for filtering and aggregate operations but may have lower performance when processing large datasets. Therefore, JOIN is the preferred approach for most real-world database applications.

Q6. Design and Implement Views for a Database System to Provide Restricted Data Access, and Justify Their Use for Security and Abstraction

Aim

To design and implement a **View** in a Student Management System that provides restricted access to data while improving security and abstraction.

Scenario

A college database stores complete student information including **StudentID, Name, Department, Marks, Phone Number, and Address**.

Faculty members only need to view **StudentID, Name, Department, and Marks**. They should **not** have access to confidential details such as Phone Number and Address.

A **View** can be created to restrict access to sensitive information.

Base Table

Student

StudentID	Name	DeptID	Marks	Phone	Address
101	Rahul	1	85	9876543210	Chennai
102	Priya	2	92	9876543211	Vellore
103	Arjun	1	78	9876543212	Salem

Step 1: Create the View

```
mysql> CREATE VIEW Student_View AS SELECT StudentID,  
->  
-> Name, DeptID, Marks FROM Student;  
Query OK, 0 rows affected (0.04 sec)
```

Step 2: Display the View

```
mysql> SELECT * FROM Student_View;  
+-----+-----+-----+-----+  
| StudentID | Name | DeptID | Marks |  
+-----+-----+-----+-----+  
| 101 | Rahul | 1 | 90 |  
| 102 | Priya | 2 | 92 |  
+-----+-----+-----+-----+  
2 rows in set (0.04 sec)
```


Step 3: Update Data Through the View

```
mysql> UPDATE Student_View
      -> SET Marks = 90
      -> WHERE StudentID = 101;
Query OK, 0 rows affected (0.02 sec)
Rows matched: 1  Changed: 0  Warnings: 0
```

Verify the Update

```
mysql> SELECT * FROM Student_View;
+-----+-----+-----+-----+
| StudentID | Name | DeptID | Marks |
+-----+-----+-----+-----+
|      101 | Rahul |      1 |    90 |
|      102 | Priya |      2 |    92 |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Justification

Security

- Sensitive information such as Phone Number and Address is hidden.
- Only authorized users can access selected columns.
- Reduces the risk of unauthorized data exposure.

Abstraction

- Users interact with only the required information.
- Complex table structures are hidden from end users.
- Makes SQL queries simpler and easier to understand.

Advantages of Views

- Provides restricted access to sensitive data.
- Improves database security.
- Simplifies complex queries.
- Supports data abstraction.
- Reduces data redundancy.
- Improves maintainability of database applications.

Applications of Views

- Student Management Systems
- Banking Systems
- Hospital Management Systems
- Employee Payroll Systems
- E-Commerce Applications

Conclusion

Views provide a secure and efficient mechanism for controlling access to database information. By exposing only the required columns, they protect confidential data while simplifying database operations through abstraction. Therefore, views play an important role in improving security, maintainability, and usability in relational database management systems.

Q7. Given a Database Schema, Write Relational Algebra Expressions and Convert Them into Equivalent SQL Queries, Analyzing Differences in Representation

Aim

To formulate relational algebra expressions based on a database schema, convert them into equivalent SQL queries, and analyze the differences between Relational Algebra and SQL.

Database Schema

Student

Attribute	Description
StudentID	Primary Key
Name	Student Name
DeptID	Department ID
Marks	Student Marks

Department

Attribute	Description
DeptID	Primary Key
DeptName	Department Name

Problem Statement

Retrieve the **StudentID**, **Name**, **Department Name**, and **Marks** of students who belong to the **CSE department** and have scored **more than 80 marks**.

Relational Algebra Expression

Step 1 – Join

$$Student \bowtie_{Student.DeptID=Department.DeptID} Department$$

Step 2 – Selection

$$\sigma_{DeptName='CSE' \wedge Marks > 80}(Student \bowtie Department)$$

Step 3 – Projection

$$\pi_{StudentID, Name, DeptName, Marks}(\sigma_{DeptName='CSE' \wedge Marks > 80}(Student \bowtie Department))$$

Equivalent SQL Query

```
mysql> SELECT S.StudentID,  
->      S.Name,  
->      D.DeptName,  
->      S.Marks  
-> FROM Student S  
-> JOIN Department D  
-> ON S.DeptID = D.DeptID  
-> WHERE D.DeptName = 'CSE'  
-> AND S.Marks > 80;  
  
+-----+-----+-----+-----+  
| StudentID | Name  | DeptName | Marks |  
+-----+-----+-----+-----+  
|      101 | Rahul | CSE      |    90 |  
+-----+-----+-----+-----+  
1 row in set (0.02 sec)
```

Analysis of Differences Between Relational Algebra and SQL

Relational Algebra	SQL
Procedural query language	Declarative query language
Specifies how to retrieve data	Specifies what data to retrieve
Uses mathematical operators such as σ , π , and $\bowtie$	Uses SQL keywords such as SELECT, FROM, WHERE, and JOIN
Mainly used for theoretical query processing	Used for practical database implementation
Does not directly manipulate database records	Supports retrieval, insertion, updating, and deletion of data

Advantages of Relational Algebra

- Provides the mathematical foundation of relational databases.
- Helps in query optimization.
- Represents database operations systematically.
- Easy to analyze query execution.

Advantages of SQL

- Easy to learn and use.
- Supports data retrieval and manipulation.
- Optimized by relational database management systems.
- Widely used in real-world database applications.

Conclusion

Relational Algebra and SQL are closely related, with SQL serving as the practical implementation of Relational Algebra concepts. While Relational Algebra describes the logical steps for data retrieval using mathematical operators, SQL provides a user-friendly syntax to perform the same operations efficiently. Understanding both representations helps in designing optimized database queries and improving query performance.

Q8. Optimize a Set of Inefficient SQL Queries by Restructuring Them Using Joins, Indexing Concepts, or Views, and Evaluate Performance Improvements

Aim

To optimize inefficient SQL queries by using **JOIN**, **Indexing**, and **Views**, and evaluate the resulting performance improvements.

Problem Statement

A Student Management System contains the following tables:

Student

StudentID	Name	DeptID	Marks
101	Rahul	1	85
102	Priya	2	92
103	Arjun	1	78
104	Sneha	3	88
105	Karthik	1	95

Department

DeptID	DeptName
1	CSE
2	ECE
3	EEE

The task is to retrieve the **Student Name** and **Department Name** efficiently.

Inefficient SQL Query (Using Subquery)

```
mysql> SELECT Name,  
-> (  
-> SELECT DeptName  
-> FROM Department D  
-> WHERE D.DeptID = S.DeptID  
-> ) AS Department  
-> FROM Student S;  
+-----+-----+  
| Name | Department |  
+-----+-----+  
| Rahul | CSE |  
| Priya | ECE |  
+-----+-----+  
2 rows in set (0.04 sec)
```

Problems

- The subquery executes for every row in the Student table.
- Performance decreases when the number of records increases.
- More database processing is required.

Optimized SQL Query (Using JOIN)

```
mysql> SELECT S.StudentID,  
-> S.Name,  
-> D.DeptName  
-> FROM Student S  
-> INNER JOIN Department D  
-> ON S.DeptID = D.DeptID;  
+-----+-----+-----+  
| StudentID | Name | DeptName |  
+-----+-----+-----+  
| 101 | Rahul | CSE |  
| 102 | Priya | ECE |  
+-----+-----+-----+  
2 rows in set (0.02 sec)
```

Advantages

- Retrieves data using a single execution plan.
- Faster execution.
- Better query optimization.
- Suitable for large databases.

Optimization Using Index

Create an index on the DeptID column.

```
mysql> CREATE INDEX idx_deptid  
-> ON Student(DeptID);  
Query OK, 0 rows affected (0.28 sec)  
Records: 0 Duplicates: 0 Warnings: 0
```

Benefits of Indexing

- Speeds up searching.
- Reduces table scanning.
- Improves JOIN performance.
- Increases query execution speed.

Optimization Using View

Create a reusable view.

```
mysql> CREATE VIEW Student_Department_View AS  
-> SELECT S.StudentID,  
->        S.Name,  
->        D.DeptName,  
->        S.Marks  
-> FROM Student S  
-> JOIN Department D  
-> ON S.DeptID = D.DeptID;  
Query OK, 0 rows affected (0.02 sec)
```

Retrieve data using the view.

```
mysql> SELECT *
      -> FROM Student_Department_View;
+-----+-----+-----+-----+
| StudentID | Name | DeptName | Marks |
+-----+-----+-----+-----+
|      101 | Rahul | CSE      | 90    |
|      102 | Priya | ECE      | 92    |
+-----+-----+-----+-----+
2 rows in set (0.04 sec)
```

Performance Evaluation

Technique	Performance	Advantage
Subquery	Low	Simple for small datasets
JOIN	High	Efficient for multiple tables
Index	Very High	Fast searching and retrieval
View	High	Simplifies complex queries and improves security

Comparison Before and After Optimization

Before Optimization	After Optimization
Repeated subquery execution	Single JOIN execution
More CPU usage	Reduced CPU usage
Slower retrieval	Faster retrieval
Less scalable	Highly scalable

Result

- The inefficient query was optimized using **JOIN**.
- An **Index** was created to improve search performance.
- A **View** was implemented for reusable and secure data access.
- Query execution became faster and more efficient.

Conclusion

SQL query optimization significantly improves database performance by reducing execution time and resource usage. Replacing repeated subqueries with **JOIN**, creating **Indexes** on frequently searched columns, and using **Views** for simplified data access enhances efficiency, scalability, and maintainability. These optimization techniques are essential for handling large databases and improving overall system performance.

Q9. Develop a Complex SQL Query Involving Grouping, Aggregation, and Nested Queries to Solve a Real-World Problem

Aim

To develop a complex SQL query using **GROUP BY**, **aggregate functions**, and **nested subqueries** to analyze student performance in a Student Management System.

Scenario

A college wants to identify departments whose **average student marks are higher than the overall average marks** of all students.

Tables

Student

StudentID	Name	DeptID	Marks
101	Rahul	1	85
102	Priya	2	92
103	Arjun	1	78
104	Sneha	3	88
105	Karthik	1	95

Department

DeptID	DeptName
1	CSE
2	ECE
3	EEE

SQL Query

```
mysql> SELECT D.DeptName,  
->      AVG(S.Marks) AS AverageMarks  
-> FROM Student S  
-> JOIN Department D  
-> ON S.DeptID = D.DeptID  
-> GROUP BY D.DeptName  
-> HAVING AVG(S.Marks) >  
-> (  
->      SELECT AVG(Marks)  
->      FROM Student  
-> );
```

```
+-----+-----+  
| DeptName | AverageMarks |  
+-----+-----+  
| ECE      |      92.0000 |  
+-----+-----+  
1 row in set (0.06 sec)
```

Explanation

JOIN

The JOIN operation combines the **Student** and **Department** tables using the common column **DeptID**.

Aggregation

The AVG() function calculates the average marks obtained by students in each department.

Grouping

GROUP BY D.DeptName groups all students according to their department.

Nested Query

```
mysql> SELECT AVG(Marks)  
-> FROM Student;  
+-----+  
| AVG(Marks) |  
+-----+  
|      91.0000 |  
+-----+  
1 row in set (0.00 sec)
```

The subquery calculates the **overall average marks** of all students.

HAVING Clause

The HAVING clause compares each department's average marks with the overall average and displays only those departments whose average is higher.

Features Used

SQL Feature	Purpose
JOIN	Combines Student and Department tables
AVG()	Calculates average marks
GROUP BY	Groups records department-wise
Nested Query	Calculates overall average marks
HAVING	Filters grouped results

Applications

- Student Performance Analysis
- Banking Transaction Reports
- Sales Performance Analysis
- Hospital Patient Statistics
- Employee Salary Analysis

Advantages

- Performs department-wise analysis efficiently.
- Uses aggregate functions to summarize data.
- Nested queries enable advanced comparisons.
- Produces meaningful reports for decision-making.

Conclusion

The SQL query successfully combines **JOIN**, **GROUP BY**, **aggregate functions**, and a **nested subquery** to solve a real-world analytical problem. It identifies departments whose average marks exceed the overall average, demonstrating how SQL can generate meaningful summaries and support effective decision-making in database applications.

Q10. Given Multiple Relations, Analyze the Query Requirements and Decide Whether to Use Views, Joins, or Subqueries, Justifying Your Choice with Reasoning

Aim

To analyze different query requirements and determine whether **Views**, **JOINS**, or **Subqueries** are the most appropriate SQL approach, with proper justification.

Scenario

A **Student Management System** contains the following relations: **Student**

StudentID	Name	DeptID	Marks
101	Rahul	1	85
102	Priya	2	92
103	Arjun	1	78
104	Sneha	3	88
105	Karthik	1	95

Department

DeptID	DeptName
1	CSE
2	ECE
3	EEE

The college requires different types of reports and secure access to student information.

Analysis of Query Requirements

Requirement 1: Display Student Name with Department Name

Best Approach: JOIN

```
mysql> SELECT S.StudentID,  
->          S.Name,  
->          D.DeptName  
-> FROM Student S  
-> JOIN Department D  
-> ON S.DeptID = D.DeptID;  
+-----+-----+-----+  
| StudentID | Name  | DeptName |  
+-----+-----+-----+  
|         101 | Rahul | CSE      |  
|         102 | Priya | ECE      |  
+-----+-----+-----+  
2 rows in set (0.00 sec)
```

Reason

- Data is stored in two related tables.
- JOIN combines matching records efficiently.
- Faster and better optimized for retrieving related information.

Requirement 2: Find Students Whose Marks Are Above the Overall Average

Best Approach: Subquery

```
mysql> SELECT Name, Marks  
-> FROM Student  
-> WHERE Marks >  
-> (  
-> SELECT AVG(Marks)  
-> FROM Student  
-> );  
+-----+-----+  
| Name  | Marks |  
+-----+-----+  
| Priya | 92    |  
+-----+-----+  
1 row in set (0.00 sec)
```

Reason

- The outer query depends on the result of the inner query.
- The subquery first calculates the average marks.
- The outer query then retrieves students scoring above that average.

Requirement 3: Provide Faculty Access Without Showing Confidential Information

Best Approach: View

```
mysql> CREATE VIEW Student_View AS
-> SELECT StudentID,
->        Name,
->        Marks
-> FROM Student;
ERROR 1050 (42S01): Table 'Student_View' already exists
```

Reason

- Views restrict access to selected columns.
- Sensitive information remains hidden.
- Provides security and data abstraction.

Comparison of SQL Approaches

Feature	JOIN	Subquery	View
Purpose	Retrieve related data from multiple tables	Use the result of another query	Provide restricted and simplified access to data
Performance	High	Moderate	High (for repeated queries)
Best Used For	Combining related tables	Filtering and aggregate calculations	Security and abstraction
Advantages	Fast, optimized, scalable	Simple for dependent queries	Protects sensitive data and simplifies queries

Evaluation

- **JOIN** is the most efficient method for retrieving data from multiple related tables because it executes using a single optimized query plan.
- **Subqueries** are useful when one query depends on the result of another, especially with aggregate functions.
- **Views** are ideal for security, abstraction, and simplifying frequently used queries by exposing only the required data.

Conclusion

The choice between **JOIN**, **Subquery**, and **View** depends on the query requirement. **JOIN** is the best option for retrieving related information from multiple tables because of its high performance and efficiency. **Subqueries** are suitable for conditional and aggregate-based retrieval where one query depends on another. **Views** provide secure and simplified access to data by hiding unnecessary or confidential information. Selecting the appropriate approach improves query performance, enhances security, and ensures efficient database management.